DP Computer Science: B 2.2.3 Stacks (Python)

---

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Understand the **LIFO principle** and stack operations
- Implement stack operations in Python using **lists**
- Analyse **performance and memory considerations**
- Choose appropriate **stack implementations**
- Apply **stacks to solve problems**

Key Vocabulary

Term	Definition
Stack	A data structure that follows the Last In, First Out (LIFO) principle
LIFO	Last In, First Out — the most recently added item is the first to be removed
Push	Adding an item to the top of the stack
Pop	Removing and returning the item from the top of the stack
Peek	Viewing the item at the top of the stack without removing it
isEmpty	Checking whether the stack is empty
Top	The element at the highest position in the stack

ATL Skills Focus

Skill	Description
Thinking	Critical thinking, creative thinking, transfer (applying stack concepts to different problems)
Communication	Collaboration, communication
Self-Management	Organisation (managing code), time management (completing tasks efficiently)

---

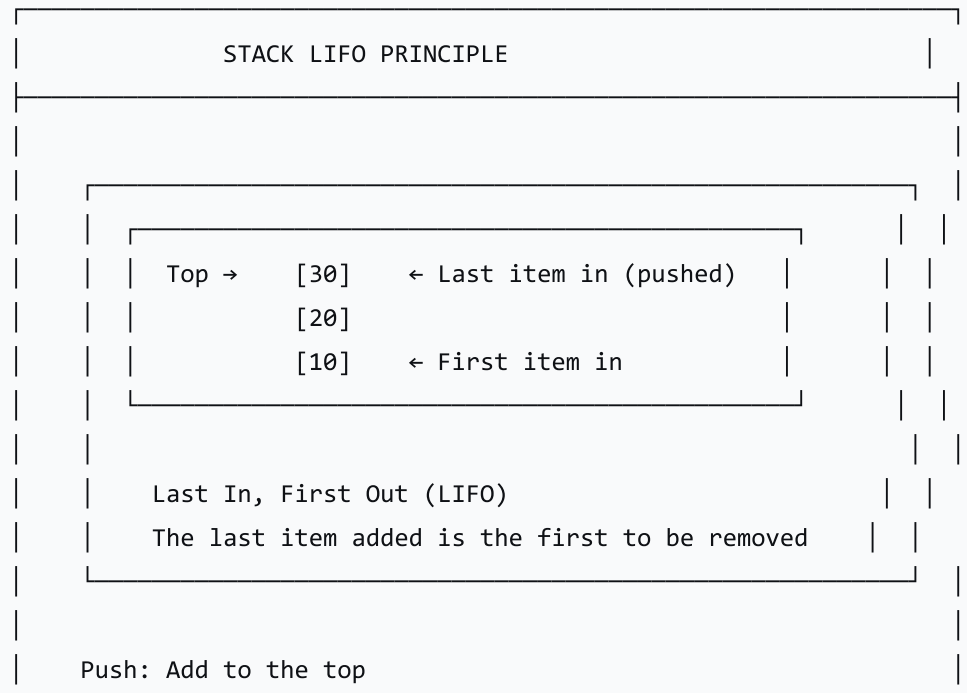
2. Introduction to Stacks

The Real-World Analogy

> *"A stack is like a stack of plates. You add plates to the top, and you remove plates from the top. The last plate placed on the stack is the first one you take off."*

The LIFO Principle

```
text
```



```
|    Pop: Remove from the top                                    |
|    Peek: View the top element without removing it              |
|                                                                |
└────────────────────────────────────────────────────────────────┘
```

Real-World Examples

Example	How a Stack Is Used
Undo in Word Processor	Each action is pushed onto a stack; undo pops the last action
Back Button in Browser	URLs are pushed onto a stack; the back button pops the last URL
Function Call Stack	Function calls are pushed; returns pop them
Stack of Plates	Plates are added and removed from the top

---

3. Stack Operations

Core Operations

Operation	Description	Python Implementation
push(item)	Adds an item to the top of the stack	`list.append(item)`
pop()	Removes and returns the item from the top	`list.pop()`
peek()	Returns the item at the top without removing	`list[-1]`
isEmpty()	Returns True if the stack is empty	`len(list) == 0`

Using Python Lists as Stacks

```python
# Creating a stack using a list
stack = []

# Push operations
stack.append(10)
stack.append(20)
stack.append(30)
print(stack)  # Output: [10, 20, 30]

# Pop operation
top = stack.pop()
print(top)    # Output: 30
print(stack)  # Output: [10, 20]

# Peek operation
top = stack[-1]
print(top)    # Output: 20
print(stack)  # Output: [10, 20]  (unchanged)

# Check if empty
if len(stack) == 0:
    print("Stack is empty")
else:
    print(f"Stack has {len(stack)} items")
```

Time Complexity

Operation	Time Complexity
Push	O(1) — constant time
Pop	O(1) — constant time
Peek	O(1) — constant time
isEmpty	O(1) — constant time

4. Implementing a Stack

Method 1: Global List (Simple)

python

```python
# Global stack data
STACK_DATA = []

def is_empty():
    """Checks if the global stack is empty."""
    return len(STACK_DATA) == 0

def push(item):
    """Adds an item to the top of the global stack."""
    global STACK_DATA
    STACK_DATA.append(item)
    print(f"Pushed: {item}")

def pop():
    """Removes and returns the item from the top of the global stack."""
    global STACK_DATA
    if is_empty():
        print("Error: Stack is empty.")
        return None
    top_item = STACK_DATA[-1]
    del STACK_DATA[-1]
    print(f"Popped: {top_item}")
    return top_item

def peek():
    """Returns the item at the top without removing it."""
    if is_empty():
        print("Error: Stack is empty.")
        return None
    return STACK_DATA[-1]

# Example usage
push(10)
push(20)
push(30)
print(f"Top item (Peek): {peek()}")
```

```
pop()  # Removes 30
pop()  # Removes 20
pop()  # Removes 10
pop()  # Error: Stack is empty
```

Method 2: OOP Implementation (Stack Class)

python

```python
class Stack:
    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def is_empty(self):
        """Check if the stack is empty."""
        return len(self.items) == 0

    def push(self, item):
        """Add an item to the top of the stack."""
        self.items.append(item)
        print(f"Pushed: {item}")

    def pop(self):
        """Remove and return the item from the top of the stack."""
        if self.is_empty():
            print("Error: Stack is empty.")
            return None
        item = self.items.pop()
        print(f"Popped: {item}")
        return item

    def peek(self):
        """Return the item at the top without removing it."""
        if self.is_empty():
            print("Error: Stack is empty.")
            return None
        return self.items[-1]

    def size(self):
        """Return the number of items in the stack."""
        return len(self.items)
```

```python
    def display(self):
        """Display all items in the stack."""
        print(f"Stack: {self.items}")

# Example usage
my_stack = Stack()
my_stack.push(5)
my_stack.push(10)
my_stack.push(15)
my_stack.display()          # Stack: [5, 10, 15]
print(my_stack.peek())      # 15
my_stack.pop()              # Removes 15
my_stack.display()          # Stack: [5, 10]
print(my_stack.size())      # 2
```

---

5. Stack Applications

Application 1: Reversing a String

python

```python
def reverse_string(text):
    """Reverse a string using a stack."""
    stack = []
    # Push all characters onto the stack
    for char in text:
        stack.append(char)
    # Pop all characters to build the reversed string
    reversed_text = ""
    while stack:
        reversed_text += stack.pop()
    return reversed_text

# Example
print(reverse_string("Hello"))  # Output: olleH
```

Application 2: Checking Balanced Parentheses

python

```python
def is_balanced(expression):
    """Check if parentheses in an expression are balanced."""
    stack = []
    opening = "([{"
    closing = ")]}"
    matching = {')': '(', ']': '[', '}': '{'}

    for char in expression:
        if char in opening:
            stack.append(char)
        elif char in closing:
            if not stack:
                return False
            if stack[-1] == matching[char]:
                stack.pop()
            else:
                return False
    return len(stack) == 0

# Examples
print(is_balanced("(a + b) * (c - d)"))    # True
print(is_balanced("(a + b) * (c - d"))     # False
print(is_balanced("{[()]}"))               # True
print(is_balanced("{[(])}"))               # False
```

Application 3: Evaluating Postfix Expressions

python

```python
def evaluate_postfix(expression):
    """Evaluate a postfix expression using a stack."""
    stack = []
    for token in expression.split():
        if token.isdigit():
            stack.append(int(token))
        else:
            b = stack.pop()
            a = stack.pop()
```

```
            if token == '+':
                stack.append(a + b)
            elif token == '-':
                stack.append(a - b)
            elif token == '*':
                stack.append(a * b)
            elif token == '/':
                stack.append(a / b)
    return stack.pop()

# Example: 3 4 + 2 * = (3 + 4) * 2 = 14
print(evaluate_postfix("3 4 + 2 *"))  # Output: 14
```

---

6. Stack Implementation Options

Python List (Array-Based)

Aspect	Description
Pros	Simple, fast O(1) operations, built-in
Cons	May need to resize (occasional O(n) cost for append)
Use When	General use; most common in Python

Linked List-Based Stack

Aspect	Description
Pros	No resizing overhead; efficient memory
Cons	More complex; extra memory for pointers
Use When	When memory is a concern

Choosing the Right Implementation

Factor	List	Linked List
Speed	Fast	Fast
Memory	May overallocate	Uses extra pointers
Complexity	Simple	More complex
Resizing	Occasionally O(n)	Never resizes

---

7. Worksheet Practice

Task 1: Basic Stack Operations

```python

# 1. Create an empty stack
# 2. Push the numbers 5, 10, 15, 20
# 3. Pop and print the top element
# 4. Peek at the top element
# 5. Check if the stack is empty
# 6. Push the number 25
# 7. Pop all elements and print them
```

Task 2: Reverse a List

```python

def reverse_list(items):
    """Reverse a list using a stack."""
    # Your code here
    pass

# Test: reverse_list([1, 2, 3, 4, 5]) → [5, 4, 3, 2, 1]
```

Task 3: Palindromes

```python
def is_palindrome(text):
    """Check if a string is a palindrome using a stack."""
    # Your code here
    pass

# Test: is_palindrome("racecar") → True
# Test: is_palindrome("hello") → False
```

---

8. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Introduction	10 min	Stack analogy; LIFO principle
Stack Operations	15 min	Push, pop, peek, isEmpty; Python implementations
Performance	5 min	Time complexity; memory considerations
Implementation	15 min	Students implement Stack class
Applications	10 min	Real-world applications; code examples
Wrap-up	5 min	Review; worksheet practice

---

9. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Implementation Activity	Assess ability to implement stack operations
Worksheet	Evaluate understanding of stack applications

10. Differentiation

Level	Strategy
Support	Provide scaffolded code examples
Challenge	Implement more complex stack applications

11. Resources

- **Presentation** (Slide Deck)
- **eBook** (B2.2.3 with examples)
- **Worksheet:** Stack operations and applications
- **Worksheet Answers**

12. Summary: Key Takeaways

Concept	Key Point
Stack	LIFO data structure
Push	Add to the top
Pop	Remove from the top

Concept	Key Point
Peek	View the top
LIFO	Last In, First Out
O(1)	All core operations are constant time

```
text

┌──────────────────────────────────────────────────────────────┐
│                    KEY REMINDERS                             │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│    ✅ Stack = LIFO data structure                            │
│    ✅ Push = add item to the top                             │
│    ✅ Pop = remove item from the top                         │
│    ✅ Peek = view top item without removing                  │
│    ✅ All operations are O(1)                                │
│    ✅ Python lists make good stacks                          │
│    ✅ Real-world uses: undo, back button, function calls     │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

> *"Stacks are one of the most fundamental data structures—they operate on the simple principle of 'last in, first out' and are used everywhere from undo buttons to function calls."*